GadgetClient – End-to-End Data Flow & Architecture

This document describes how the Client Admin GUI, Client Database (v15), and Mapping GUI v13.1 work together to ingest, standardise, validate, invoice, and report Gadget client data. It also includes the final recommended patch plan to fully connect Admin-authored mappings into the processing pipeline.

1. System Overview

The GadgetClient system is designed to process client-provided CSV files through a controlled, auditable pipeline.

Its goals are:

• Canonical standardisation of heterogeneous client data

• Robust validation and error isolation via failure packs

• Repeatable invoicing and bordereau reporting

• Separation of configuration (clients, rules, mappings) from processing logic

The system is composed of three major layers:

1) Client Admin GUI (authoring)

2) Client Database v15 (runtime resolution)

3) Mapping GUI v13.1 (processing engine)

2. Client Admin GUI (Authoring Layer)

The Client Admin GUI is the authoritative authoring tool for client-specific configuration.

It writes all configuration into:

C:\GadgetClient\config\client_db.json

Each client record contains:

• Client identity (client_id, client_name, product)

• Name aliases for matching and inference

• Excess rules (allowed values and default)

• Email recipients (TO / CC)

• Mapping tables:

- template_fields (raw header → canonical field)

- value_maps.cover_level

- value_maps.cover_type

All mapping keys are normalised to ensure deterministic lookups.

The Admin GUI never processes files; it only defines behaviour.

3. Client Database v15 (Runtime Resolution Layer)

The Client Database v15 module provides a safe, backward-compatible runtime API over client_db.json.

Primary responsibility:

• Resolve client identity deterministically without breaking legacy inference

Key function:

resolve_client_identity(...)

Resolution order:

1) Trust inferred client_id if it exists in DB

2) Exact match on client_name

3) Alias match

4) Conservative fuzzy match (threshold + gap)

5) Fallback to legacy v13 inference

The resolver returns a structured ClientResolution object containing:

• found flag

• canonical client_id and client_name

• resolution source

• confidence score

In v13.1 today, this resolver is used primarily for email canonicalisation.

4. Mapping GUI v13.1 (Processing Layer)

Mapping GUI v13.1 is the execution engine for both GUI-driven and headless processing.

Capabilities:

• Header auto-detection and delimiter inference

• Auto-suggested header mapping using synonyms + similarity

• Manual mapping via GUI

• Mapping persistence to JSON

• Workflow 1: Clean CSV export

• Workflow 2: Invoicing, micro, backup, control reports

• Workflow 3: Bordereau (per-client and master)

• Inbox watcher, retry system, and failure packs

• Email notifications

All processing is driven by contracts rather than ad-hoc paths.

5. Inbox Contract & File Lifecycle

Input base:

C:\GadgetClient\in\

Expected structure:

Inbox\YYYY-MM\file.csv (preferred)

Inbox\file.csv (fallback)

Processing lifecycle:

• File detected in Inbox

• Validated and processed

• On success: moved to Processed\YYYY-MM\timestamp__file.csv

• On failure: moved into a Failure Pack

Supported input format:

• CSV only

6. Failure Packs (Audit & Recovery)

Failure packs are immutable audit artifacts.

Structure:

Failed\\YYYY-MM\\

• original CSV

• failure_summary.txt

• failure_details.json

• detected_headers.txt

• mapping_used.json

• retry history (if applicable)

Failure categories:

• MissingMapping

• HeaderMismatch

• BadEncoding

• ValidationErrors

• UnknownCoverValues

• UnexpectedError

Failure packs allow safe diagnosis and controlled retries.

7. End-to-End Processing Flow

1) File is detected in Inbox

2) Invoice month is inferred (folder → config → default)

3) Headers, delimiter, and encoding are detected

4) Header mapping is auto-suggested

5) Existing mapping JSON is loaded if available

6) Minimum required fields are enforced

7) Row-level validation is executed

8) Workflow 1 Clean CSV is written

9) Optional Workflow 2 outputs are generated

10) Optional Workflow 3 bordereau outputs are generated

11) Original file is archived to Processed

12) Emails are sent (success or failure)

8. Current Use of Client Database in v13.1

At present, client_db_v15 is used only as a safe overlay for email identity.

Usage points:

• Success email: canonical client name resolution

• Failure email: canonical client name (and sometimes ID) resolution

The database does NOT yet drive:

• Header mappings

• Value normalisation

• Output naming

This design choice ensures zero regression risk while introducing the DB.

9. Final Connect Patch Plan (Recommended)

Objective:

Make Admin GUI mappings authoritative while preserving v13 stability.

Step 1 – Extend client_db_v15 accessors:

• Add functions to read Admin GUI schema:

- template_fields

- value_maps.cover_level

- value_maps.cover_type

Step 2 – Early client resolution:

• In process_single_inbox_file(), resolve client identity immediately after header detection.

Step 3 – Mapping source priority:

1) Client DB mapping tables (if client resolved)

2) Existing per-client mapping JSON

3) Auto-suggest mapping

Step 4 – Value normalisation:

• Replace global COVERLEVEL_VALUE_MAP / COVERTYPE_VALUE_MAP with DB-provided maps when available.

Step 5 – Backward compatibility:

• If DB mapping is missing or client is unresolved, fall back to existing behaviour.

• No breaking changes to v13 workflows or folder contracts.

Result:

• Admin GUI becomes the single source of truth.

• Mapping GUI becomes a pure executor.

• Client DB becomes the authoritative configuration layer.

10. Architectural Outcome

After the final patch:

• Client Admin GUI defines behaviour

• Client Database enforces identity and rules

• Mapping GUI executes workflows deterministically

• All outputs remain traceable, auditable, and retryable

This architecture supports future products, additional workflows, and full operational automation.